Task 9 — Learn Useful Git Recovery Commands

DEVOPS-005: Git Advanced Workflow & Team Collaboration

Important Rule

Do not blindly run destructive Git commands. You must understand what each command does before using it — a wrong reset or restore can permanently delete your work.

Commands Explained

`git diff`

Shows changes in your working directory that are NOT yet staged. It compares your current file contents against the last commit, for everything you have edited but not yet added with git add. Safe — read-only, no changes made.

`git diff --staged`

Shows changes that ARE staged (added with git add) but not yet committed. It compares the staging area against the last commit, so you can review exactly what will go into your next commit. Safe — read-only, no changes made.

`git restore`

Discards uncommitted changes in the working directory for the named file, reverting it back to the last committed version. DESTRUCTIVE — any unsaved edits to that file are permanently lost. Use when you want to throw away local edits and start fresh from the last commit.

`git reset`

Unstages files — it moves changes from the staging area back to the working directory without discarding the actual edits (soft/mixed reset). With --hard it also discards the working directory changes and moves the branch pointer back, permanently deleting commits. DESTRUCTIVE with --hard. Rewrites history — dangerous on shared/pushed branches.

`git revert`

Creates a NEW commit that undoes the changes introduced by a specified previous commit. It does NOT rewrite history — it adds a new entry on top. Safe to use on shared or pushed branches because the history is preserved and teammates can still pull without conflicts.

restore vs reset vs revert — Key Differences

Command	Scope	Rewrites History?	Safe on Shared Branch?	Use When...
`git restore`	Working directory / staging area	No	Yes	You want to discard uncommitted file edits
`git reset`	Staging area and/or commit history	Yes (with --hard)	No (dangerous)	You want to unstage or undo local commits before pushing
`git revert`	Commit history (adds new commit)	No	Yes	You need to undo a pushed commit without breaking others' work

Golden Rule: If you have already pushed to a shared remote, prefer git revert over git reset. Resetting pushed history forces others to re-sync and can cause conflicts and data loss for your team.